

Approches classiques de l'IA

Chapitre 7 : Inférence en Logique du Premier Ordre

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 7. Inférence en Logique du Premier Ordre

- 1 Logique du premier ordre
- 2 Utilisation de la Logique du premier ordre
- 3 Inférence en Logique du premier ordre
- 4 Unification et lifting
- 5 Forward chaining
- 6 Backward chaining et Programmation logique
- 7 Preuves par Résolution
- 8 Résumé du chapitre

Logique du Premier Ordre

Chapitre 6 : agent basé sur la connaissance qui utilise la logique propositionnelle pour représenter le monde et en déduire les actions à prendre.

Mais la logique propositionnelle a une expressivité très limitée.

Exemple (Le monde de Wumpus)

En LP, on ne peut pas dire simplement “les trous provoquent des courant d’air dans les cases adjacentes”. Il faudrait écrire une phrase spécifique pour chaque case ! Exemple, pour la case de départ :

$$B_{1,1} \Leftrightarrow (P_{1,2} \vee P_{2,1})$$

Dans ce chapitre, on va utiliser une forme plus expressive de logique : la logique du premier ordre (**First Order Logic** – FOL) appelée aussi le “calcul des prédicats (du premier ordre)”.

Logique du premier ordre

Logique du Premier Ordre – FOL

Là où la LP fait l'hypothèse que le monde contient des **faits**, la logique du premier ordre (comme le langage naturel) fait l'hypothèse que le monde contient des objets et des relations :

- **Objets** : personnes, maisons, nombres, théories, Linus Torvalds, couleurs, matchs de rugby, guerres, siècles, ...
- **Relations** :
 - ▶ (unaires $\simeq$ **propriétés** des objets) rouge, rond, buggé, premier, carnivore, ...
 - ▶ (n -aires $\simeq$ relations entre n objets) frère de, plus grand que, à l'intérieur de, partie de, a comme couleur, se produit après, appartient à, ...

Logique du Premier Ordre – FOL

Un type particulier de relations : les **fonctions** où il n'y a qu'une seule "valeur" possible pour une entrée donnée.

- **Fonctions** : père de, meilleur ami de, deuxième manche de, un de plus que, à la fin de, ...

Expressivité puissante : presque n'importe quelle assertion peut être vue comme référant à des objets et des propriétés ou des relations.

Syntaxe de FOL : éléments de base

Constantes	<i>John, 2, UMONS,...</i>
Prédicats	<i>Brother, >,...</i>
Fonctions	<i>Sqrt, LeftLegOf,...</i>

Syntaxe de FOL : éléments de base

Constantes	<i>John, 2, UMONS,...</i>
Prédicats	<i>Brother, >,...</i>
Fonctions	<i>Sqrt, LeftLegOf,...</i>

Ces éléments forment le **vocabulaire** ou les **symboles** et vont être combinés pour former des **termes**, **phrases atomiques** et **phrases complexes** grâce aux connecteurs, à l'égalité, aux variables et aux quantificateurs.

Variables	<i>x, y, a, b,...</i>
Connecteurs	$\wedge, \vee, \neg, \Rightarrow, \Leftrightarrow$
Egalité	$=$
Quantificateurs	$\forall, \exists$

Remarque : dans certains cas l'égalité n'est pas incluse dans la syntaxe de FOL. On peut dire "Logique du premier ordre avec égalité" pour éviter toute ambiguïté (elle est incluse dans le cadre de ce cours).

Syntaxe : Termes

Un terme correspond à un objet

Terme = *constante* ou *variable*
ou *fonction*(*terme*₁, ..., *terme*_n)

Exemples

John,
x,
Sqrt(*x*),
LeftLegOf(*John*)

Convention : les symboles (constants, prédicats et fonctions) commencent par une majuscule ; les variables par une minuscule.

Syntaxe : Phrases atomiques

Une phrase atomique correspond à une relation entre objets

Phrase atomique = $\text{predicat}(\text{terme}_1, \dots, \text{terme}_n)$
ou $\text{terme}_1 = \text{terme}_2$

Exemples

$\text{Brother}(\text{John}, \text{Richard}) ;$

$\text{Sqrt}(x) = 2 ;$

$> (\text{Length}(\text{LeftLegOf}(\text{Richard})), \text{Length}(\text{LeftLegOf}(\text{John})))$

Syntaxe : Phrases complexes

Des phrases complexes sont construites à partir d'autres phrases (atomiques ou complexes) en utilisant des connecteurs

$$\neg S, \quad S_1 \wedge S_2, \quad S_1 \vee S_2, \quad S_1 \Rightarrow S_2, \quad S_1 \Leftrightarrow S_2$$

Exemples

$$\begin{aligned} & \textit{Sibling}(\textit{John}, \textit{Richard}) \Rightarrow \textit{Sibling}(\textit{Richard}, \textit{John}) ; \\ & >(1,2) \vee \leq(1,2) ; \\ & >(1,2) \wedge \neg >(1,2) \end{aligned}$$

Remarque : “sibling” = issus des mêmes parents

Syntaxe : Phrases complexes

Remarque : les opérateurs logiques ne s'appliquent pas sur les termes (**objets**) mais sur les phrases (**relations**).

Exemple

$In(Paris \wedge Lille, France)$ n'est pas correct.

Avant de parler des quantificateurs et des variables, voyons déjà ce qu'est un **modèle** en FOL.

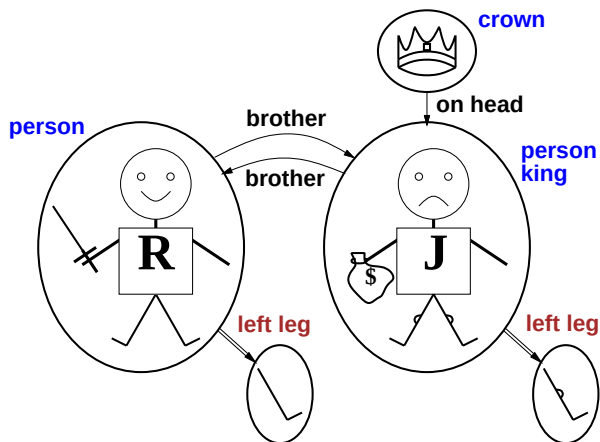
Modèles en logique du premier ordre (FOL)

Un **modèle** contient

- ≥ 1 objets (= **éléments du domaine**) et
- les relations entre les objets.

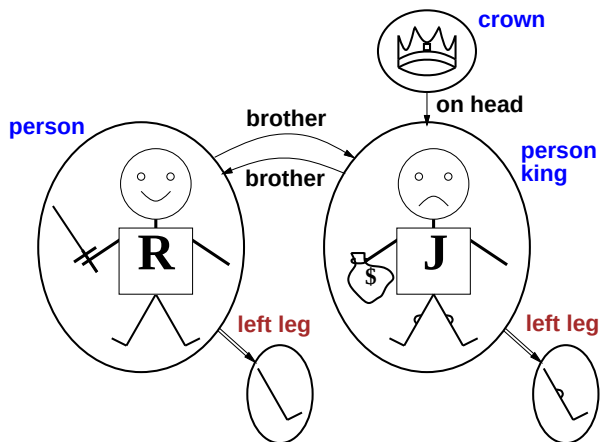
Question / rappel : comment est défini un modèle en Logique Propositionnelle ?

Modèles pour FOL : exemple



Objets et relations ??

Modèles pour FOL : exemple



Objets et relations ?? Ce modèle contient 5 objets (ovales), 2 relations binaires (en noir), 3 relations unaires ou propriétés (en bleu) et 1 fonction unaire (en rouge).

Vérité en logique du premier ordre (FOL)

- Les phrases sont vraies par rapport à un **modèle** et une **interprétation** : comme en LP, tout modèle doit fournir l'information nécessaire pour déterminer si une phrase est vraie. Donc, en plus des objets et des relations, chaque modèle inclut une interprétation.
- L'interprétation spécifie des référents pour
 - ▶ les **symboles constants** → **objets** ;
 - ▶ les **symboles de predicats** → **relations** ;
 - ▶ les **symboles de fonctions** → **relations fonctionnelles**.
- Une phrase atomique

$\text{predicat}(\text{terme}_1, \dots, \text{terme}_n)$

est vraie ssi les **objets** référés par *$\text{terme}_1, \dots, \text{terme}_n$* sont dans une **relation** référée par *predicat* .

- En ce qui concerne les connecteurs logiques : même syntaxe et sémantique qu'en LP.

Vérité en logique du premier ordre (FOL)

- Les phrases sont vraies par rapport à un **modèle** et une **interprétation** : comme en LP, tout modèle doit fournir l'information nécessaire pour déterminer si une phrase est vraie. Donc, en plus des objets et des relations, chaque modèle inclut une interprétation.
- L'interprétation spécifie des référents pour
 - ▶ les **symboles constants** → **objets** ;
 - ▶ les **symboles de predicats** → **relations** ;
 - ▶ les **symboles de fonctions** → **relations fonctionnelles**.
- Une phrase atomique

predicat($terme_1, \dots, terme_n$)

est vraie ssi les **objets** référés par *terme₁, ..., terme_n* sont dans une **relation** référée par *predicat*.

- En ce qui concerne les connecteurs logiques : même syntaxe et sémantique qu'en LP.

Vérité en logique du premier ordre (FOL)

- Les phrases sont vraies par rapport à un **modèle** et une **interprétation** : comme en LP, tout modèle doit fournir l'information nécessaire pour déterminer si une phrase est vraie. Donc, en plus des objets et des relations, chaque modèle inclut une interprétation.
- L'interprétation spécifie des référents pour
 - ▶ les **symboles constants** → **objets** ;
 - ▶ les **symboles de predicats** → **relations** ;
 - ▶ les **symboles de fonctions** → **relations fonctionnelles**.
- Une phrase atomique

$$\textit{predicat}(\textit{terme}_1, \dots, \textit{terme}_n)$$

est vraie ssi les **objets** référés par $\textit{terme}_1, \dots, \textit{terme}_n$ sont dans une **relation** référée par *predicat*.

- En ce qui concerne les connecteurs logiques : même syntaxe et sémantique qu'en LP.

Vérité en logique du premier ordre (FOL)

- Les phrases sont vraies par rapport à un **modèle** et une **interprétation** : comme en LP, tout modèle doit fournir l'information nécessaire pour déterminer si une phrase est vraie. Donc, en plus des objets et des relations, chaque modèle inclut une interprétation.
- L'interprétation spécifie des référents pour
 - ▶ les **symboles constants** → **objets** ;
 - ▶ les **symboles de predicats** → **relations** ;
 - ▶ les **symboles de fonctions** → **relations fonctionnelles**.
- Une phrase atomique

$$\textit{predicat}(\textit{terme}_1, \dots, \textit{terme}_n)$$

est vraie ssi les **objets** référés par $\textit{terme}_1, \dots, \textit{terme}_n$ sont dans une **relation** référée par *predicat*.

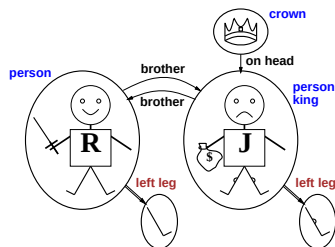
- En ce qui concerne les connecteurs logiques : même syntaxe et sémantique qu'en LP.

Exemple de vérité

Soit l'interprétation suivante :

- *Richard* → Richard Cœur de Lion
- *John* → le méchant Roi Jean
- *Brother* → relation de fraternité

Selon cette interprétation,
Brother(Richard, John) est vrai dans le
 cas où Richard Cœur de Lion et le
 méchant Roi Jean sont dans une relation
 de fraternité dans le modèle.



Symboles et interprétation : quelques remarques

- D'une manière formelle, une relation est un ensemble de **tuples** d'objets qui sont reliés. Ainsi, la **relation de fraternité** à laquelle *Brother* réfère peut être définie dans le modèle précédent par :

$$\{ \langle \text{Richard Cœur de Lion}, \text{le Roi Jean} \rangle, \\ \langle \text{le Roi Jean}, \text{Richard Cœur de Lion} \rangle \}$$

- il y a quantité d'interprétations possibles (même "farfelues"). Par exemple une interprétation associe *Richard* à la couronne et *John* à la jambe gauche du Roi Jean.

Symboles et interprétation : quelques remarques

- D'une manière formelle, une relation est un ensemble de **tuples** d'objets qui sont reliés. Ainsi, la **relation de fraternité** à laquelle *Brother* réfère peut être définie dans le modèle précédent par :

$$\{ \langle \text{Richard Cœur de Lion}, \text{le Roi Jean} \rangle, \\ \langle \text{le Roi Jean}, \text{Richard Cœur de Lion} \rangle \}$$

- il y a quantité d'interprétations possibles (même "farfelues"). Par exemple une interprétation associe *Richard* à la couronne et *John* à la jambe gauche du Roi Jean.

Symboles et interprétation : quelques remarques

- Tous les objets ne doivent pas “avoir un nom” (être associés à un symbole). Par exemple, l'interprétation donnée précédemment ne nomme pas la couronne ou les jambes.
- Il est également possible pour un objet d'avoir plusieurs noms (e.g., il y a une interprétation où *Richard* et *John* réfèrent à la couronne).
- Si vous trouvez les remarques précédentes troublantes : cf. la LP (Chap. 6) où un modèle possible est d'avoir 2 symboles de proposition *Nuageux* et *Ensoleille* qui sont vrais : c'est le rôle de la KB de rejeter les modèles qui sont inconsistants avec notre connaissance.

Syntaxe : Quantificateurs et égalité

Il reste des éléments particuliers à présenter : les **quantificateurs** $\forall$ et $\exists$; et l'**égalité**.

Les quantificateurs comblent un manque important de l'expressivité de la LP : ils permettent d'exprimer des propriétés pour des collections entières d'objets (plutôt que de les énumérer tous par leur nom).

Syntaxe : une phrase complexe peut également être définie par

$$\textit{Quantificateur Variable}, \dots S$$

où S est une phrase complexe ou atomique.

Quantification universelle : $\forall$

Syntaxe : $\forall \langle \text{variables} \rangle \langle \text{phrase} \rangle$

Exemple

“Tout le monde à Mons est intelligent” :

$$\forall x \text{ At}(x, \text{Mons}) \Rightarrow \text{Smart}(x)$$

Sémantique : $\forall x \text{ } P$ est vrai dans un modèle m ssi P est vrai pour tout objet possible x dans le modèle.

Quantification universelle : $\forall$

Syntaxe : $\forall \langle \text{variables} \rangle \langle \text{phrase} \rangle$

Exemple

“Tout le monde à Mons est intelligent” :

$$\forall x \text{ At}(x, \text{Mons}) \Rightarrow \text{Smart}(x)$$

Sémantique : $\forall x \text{ } P$ est vrai dans un modèle m ssi P est vrai pour tout objet possible x dans le modèle.

Intuition : conjonction de toutes les instanciations possibles de P

$$\begin{aligned} & (\text{At}(\text{John}, \text{Mons}) \Rightarrow \text{Smart}(\text{John})) \\ \wedge & (\text{At}(\text{Richard}, \text{Mons}) \Rightarrow \text{Smart}(\text{Richard})) \\ \wedge & (\text{At}(\text{Mons}, \text{Mons}) \Rightarrow \text{Smart}(\text{Mons})) \\ \wedge & \dots \end{aligned}$$

Quantification universelle : une erreur fréquente à éviter

Typiquement, $\Rightarrow$ est le principal connecteur avec $\forall$

Erreur fréquente : utiliser $\wedge$ comme connecteur principal avec $\forall$:

$$\forall x \text{ } At(x, \textit{Mons}) \wedge \textit{Smart}(x)$$

signifie ...

Quantification universelle : une erreur fréquente à éviter

Typiquement, $\Rightarrow$ est le principal connecteur avec $\forall$

Erreur fréquente : utiliser $\wedge$ comme connecteur principal avec $\forall$:

$$\forall x \text{ } At(x, \textit{Mons}) \wedge \textit{Smart}(x)$$

signifie ...

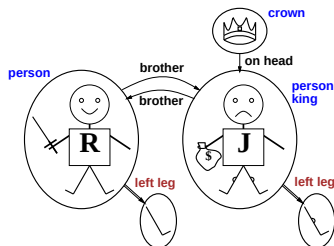
“Tout le monde est à Mons et tout le monde est intelligent”

Quantification universelle : exemple

- Soit la phrase “Tous les rois sont des personnes”
- En FOL, cela donne :

$$\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$$

- Formellement, $\forall x P$ est vrai dans un modèle si P est vrai pour toutes les interprétations possibles étendues du modèle, c.-à-d. que chaque interprétation spécifie un élément du domaine auquel x réfère.

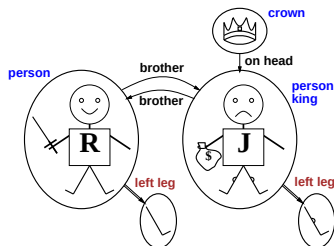


Quantification universelle : exemple

- Soit la phrase “Tous les rois sont des personnes”
- En FOL, cela donne :

$$\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$$

- Formellement, $\forall x P$ est vrai dans un modèle si P est vrai pour toutes les interprétations possibles étendues du modèle, c.-à-d. que chaque interprétation spécifie un élément du domaine auquel x réfère.

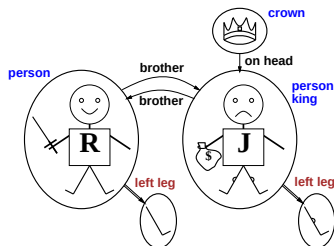


Quantification universelle : exemple

- Soit la phrase “Tous les rois sont des personnes”
- En FOL, cela donne :

$$\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$$

- Formellement, $\forall x P$ est vrai dans un modèle si P est vrai pour toutes les interprétations possibles étendues du modèle, c.-à-d. que chaque interprétation spécifie un élément du domaine auquel x réfère.



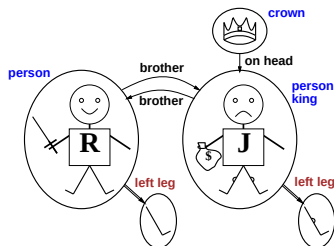
- $x \rightarrow$ Richard cœur de lion
- $x \rightarrow$ Le Roi Jean
- $x \rightarrow$ La jambe gauche de Richard
- $x \rightarrow$ La jambe gauche de Jean
- $x \rightarrow$ La couronne

Quantification universelle : exemple

- Soit la phrase “Tous les rois sont des personnes”
- En FOL, cela donne :

$$\forall x \text{ King}(x) \Rightarrow \text{Person}(x)$$

- Formellement, $\forall x P$ est vrai dans un modèle si P est vrai pour toutes les interprétations possibles étendues du modèle, c.-à-d. que chaque interprétation spécifie un élément du domaine auquel x réfère.



- $x \rightarrow$ Richard cœur de lion
- $x \rightarrow$ Le Roi Jean
- $x \rightarrow$ La jambe gauche de Richard
- $x \rightarrow$ La jambe gauche de Jean
- $x \rightarrow$ La couronne

On peut étendre l'interprétation de 5 manières : $\text{King}(x) \Rightarrow \text{Person}(x)$ est toujours vrai.

Quantification existentielle : $\exists$

Syntaxe : $\exists \langle \text{variables} \rangle \langle \text{phrase} \rangle$

Exemple

“Quelqu’un à Mons est intelligent” :

$$\exists x \text{ At}(x, \text{Mons}) \wedge \text{Smart}(x)$$

Sémantique : $\exists x P$ est vrai dans un modèle m ssi P est vrai pour au moins un objet x dans le modèle.

Quantification existentielle : $\exists$

Syntaxe : $\exists \langle \text{variables} \rangle \langle \text{phrase} \rangle$

Exemple

“Quelqu’un à Mons est intelligent” :

$$\exists x \text{ At}(x, \text{Mons}) \wedge \text{Smart}(x)$$

Sémantique : $\exists x P$ est vrai dans un modèle m ssi P est vrai pour au moins un objet x dans le modèle.

Intuition : disjonction de toutes les instantiations possibles de P

$$\begin{aligned} & (\text{At}(\text{John}, \text{Mons}) \wedge \text{Smart}(\text{John})) \\ \vee & (\text{At}(\text{Richard}, \text{Mons}) \wedge \text{Smart}(\text{Richard})) \\ \vee & (\text{At}(\text{Mons}, \text{Mons}) \wedge \text{Smart}(\text{Mons})) \\ \vee & \dots \end{aligned}$$

Quantification existentielle : une autre erreur fréquente à éviter

Typiquement, $\wedge$ est le principal connecteur avec $\exists$

Erreur fréquente : utiliser $\Rightarrow$ comme connecteur principal avec $\exists$:

$$\exists x \text{ } At(x, Mons) \Rightarrow Smart(x)$$

est vrai si ...

Quantification existentielle : une autre erreur fréquente à éviter

Typiquement, $\wedge$ est le principal connecteur avec $\exists$

Erreur fréquente : utiliser $\Rightarrow$ comme connecteur principal avec $\exists$:

$$\exists x \text{ } At(x, Mons) \Rightarrow Smart(x)$$

est vrai si ...

“Il y a quelqu'un qui n'est pas à Mons ou il existe quelqu'un d'intelligent !”

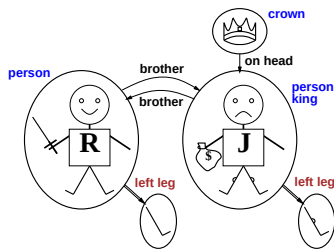
Rappel : $S_1 \Rightarrow S_2$ est vrai ssi S_1 est faux ou S_2 est vrai.

Quantification existentielle : exemple

- Soit la phrase “Le Roi Jean a une couronne sur sa tête”
- En FOL, cela donne :

$$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

- Formellement, $\exists x P$ sera vrai dans un modèle si P est vrai pour au moins une interprétation *étendue*, qui assigne un élément du domaine à x .

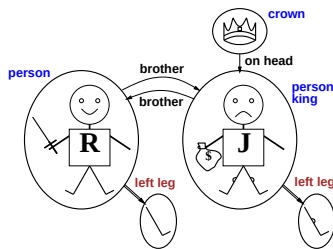


Quantification existentielle : exemple

- Soit la phrase “Le Roi Jean a une couronne sur sa tête”
- En FOL, cela donne :

$$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

- Formellement, $\exists x P$ sera vrai dans un modèle si P est vrai pour au moins une interprétation *étendue*, qui assigne un élément du domaine à x .

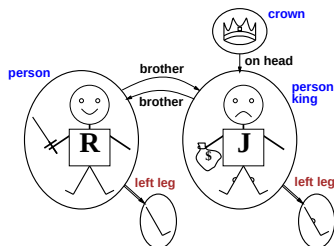


Quantification existentielle : exemple

- Soit la phrase “Le Roi Jean a une couronne sur sa tête”
- En FOL, cela donne :

$$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

- Formellement, $\exists x P$ sera vrai dans un modèle si P est vrai pour au moins une interprétation *étendue*, qui assigne un élément du domaine à x .



- $x \rightarrow$ Richard cœur de lion
- $x \rightarrow$ Le Roi Jean
- $x \rightarrow$ La jambe gauche de Richard
- $x \rightarrow$ La jambe gauche de Jean
- $x \rightarrow$ La couronne

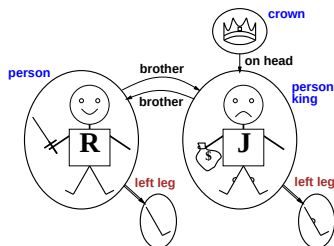
Quantification existentielle : exemple

- Soit la phrase “Le Roi Jean a une couronne sur sa tête”
- En FOL, cela donne :

$$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$$

- Formellement, $\exists x P$ sera vrai dans un modèle si P est vrai pour au moins une interprétation **étendue**, qui assigne un élément du domaine à x .

Dans ce modèle, $\text{Crown}(x) \wedge \text{OnHead}(x, \text{John})$ est vrai quand $x \rightarrow$ **La couronne**.



- $x \rightarrow$ Richard cœur de lion
- $x \rightarrow$ Le Roi Jean
- $x \rightarrow$ La jambe gauche de Richard
- $x \rightarrow$ La jambe gauche de Jean
- $x \rightarrow$ La couronne

Propriétés des quantificateurs

- $\forall x \forall y$ est équivalent à $\forall y \forall x$ (on peut l'écrire $\forall x, y$).

Exemples

“Des frères sont issus des mêmes parents” :

$$\forall x \forall y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$$

“La fraternité est une relation symétrique”

$$\forall x, y \text{ Brother}(x, y) \Leftrightarrow \text{Brother}(y, x)$$

- $\exists x \exists y$ est équivalent à $\exists y \exists x$ (on peut l'écrire $\exists x, y$).

Propriétés des quantificateurs

- $\forall x \forall y$ est équivalent à $\forall y \forall x$ (on peut l'écrire $\forall x, y$).

Exemples

“Des frères sont issus des mêmes parents” :

$$\forall x \forall y \text{ Brother}(x, y) \Rightarrow \text{Sibling}(x, y)$$

“La fraternité est une relation symétrique”

$$\forall x, y \text{ Brother}(x, y) \Leftrightarrow \text{Brother}(y, x)$$

- $\exists x \exists y$ est équivalent à $\exists y \exists x$ (on peut l'écrire $\exists x, y$).

Propriétés des quantificateurs

- $\exists x \forall y$ n'est pas équivalent à $\forall y \exists x$

Propriétés des quantificateurs

- $\exists x \forall y$ n'est pas équivalent à $\forall y \exists x$

Exemples

$\exists x \forall y \text{ Loves}(x, y) :$

$\forall y \exists x \text{ Loves}(x, y) :$

$\exists y \forall x \text{ Loves}(x, y) :$

$\forall x \exists y \text{ Loves}(x, y) :$

Propriétés des quantificateurs

- $\exists x \forall y$ n'est pas équivalent à $\forall y \exists x$

Exemples

$\exists x \forall y \text{ Loves}(x, y) :$

“Il y a une personne qui aime tout le monde”

$\forall y \exists x \text{ Loves}(x, y) :$

$\exists y \forall x \text{ Loves}(x, y) :$

$\forall x \exists y \text{ Loves}(x, y) :$

Propriétés des quantificateurs

- $\exists x \forall y$ n'est pas équivalent à $\forall y \exists x$

Exemples

$\exists x \forall y \text{ Loves}(x, y) :$

“Il y a une personne qui aime tout le monde”

$\forall y \exists x \text{ Loves}(x, y) :$

“Chaque personne est aimée par au moins une personne”

$\exists y \forall x \text{ Loves}(x, y) :$

$\forall x \exists y \text{ Loves}(x, y) :$

Propriétés des quantificateurs

- $\exists x \forall y$ n'est pas équivalent à $\forall y \exists x$

Exemples

$\exists x \forall y \text{ Loves}(x, y) :$

“Il y a une personne qui aime tout le monde”

$\forall y \exists x \text{ Loves}(x, y) :$

“Chaque personne est aimée par au moins une personne”

$\exists y \forall x \text{ Loves}(x, y) :$

“Il y a quelqu'un qui est aimé par tout le monde”

$\forall x \exists y \text{ Loves}(x, y) :$

Propriétés des quantificateurs

- $\exists x \forall y$ n'est pas équivalent à $\forall y \exists x$

Exemples

$\exists x \forall y \text{ Loves}(x, y) :$

“Il y a une personne qui aime tout le monde”

$\forall y \exists x \text{ Loves}(x, y) :$

“Chaque personne est aimée par au moins une personne”

$\exists y \forall x \text{ Loves}(x, y) :$

“Il y a quelqu'un qui est aimé par tout le monde”

$\forall x \exists y \text{ Loves}(x, y) :$

“Chacun aime quelqu'un”

Propriétés des quantificateurs

Dualité des quantificateurs : $\forall$ peut être exprimé en utilisant $\exists$ et vice-versa

Propriétés des quantificateurs

Dualité des quantificateurs : $\forall$ peut être exprimé en utilisant $\exists$ et vice-versa

Exemple

$$\forall x \text{ Likes}(x, \text{IceCream}) \longrightarrow \neg(\exists x \neg \text{Likes}(x, \text{IceCream}))$$

$$\exists x \text{ Likes}(x, \text{Broccoli}) \longrightarrow \neg(\forall x \neg \text{Likes}(x, \text{Broccoli}))$$

Egalité

Syntaxe et sémantique : $terme_1 = terme_2$ est vrai selon une interprétation donnée ssi $terme_1$ et $terme_2$ réfèrent le même objet

Exemples

$1 = 2$ et $\forall x \times (Sqrt(x), Sqrt(x)) = x$ sont satisfaisables (vraies dans certains modèles)

$2 = 2$ est valide (tautologie, vraie dans tous les modèles)

Utilisation de la Logique du premier ordre

Exemple d'utilisation de FOL

On a vu quelques exemple d'utilisation de FOL mais celle-ci est suffisamment puissante que pour exprimer par exemple la théorie des ensembles (où $\cup$ et $\cap$ deviennent des fonctions binaires) ou la notion de liste.

On peut donc utiliser des ensembles ou des listes en FOL ! Dans ce qui suit nous utiliserons une syntaxe similaire à Python pour exprimer une liste. Exemple : $[A, B, C]$

Illustrons l'expressivité de FOL dans le monde de Wumpus.

Wumpus : KB exprimée en FOL

Au chap. 6, nous avons utilisé la LP pour exprimer quelques axiomes sur le monde de Wumpus.

Nous allons ici donner des axiomes en FOL qui seront beaucoup plus concis pour décrire les perceptions, les actions, l'environnement, etc.

Wumpus — FOL : perceptions et actions

Rappel : un agent, dans le monde de Wumpus, reçoit un vecteur de perceptions avec 5 éléments (Stench, Breeze, Glitter, Bump, Scream).

Wumpus — FOL : perceptions et actions

Rappel : un agent, dans le monde de Wumpus, reçoit un vecteur de perceptions avec 5 éléments (Stench, Breeze, Glitter, Bump, Scream).

Une phrase correspondante en FOL pourrait également inclure le moment (temps exprimé par un entier) où les perceptions ont été reçues, pour éviter toute confusion.

Percept([*Stench*, *Breeze*, *Glitter*, *None*, *None*], 5)

Ici, *Percept* est un prédicat binaire, et *Stench* etc. sont des constantes placées dans une liste.

Wumpus — FOL : perceptions et actions

Les **actions** dans le monde de Wumpus peuvent être représentées par des **termes** :

Turn(Left), Turn(Right), Forward, Shoot, Grab, Climb.

Wumpus — FOL : perceptions et actions

Les **actions** dans le monde de Wumpus peuvent être représentées par des **termes** :

Turn(Left), Turn(Right), Forward, Shoot, Grab, Climb.

Exemple : à l'instant $t = 5$, l'agent perçoit une odeur, un courant d'air et un scintillement.

Tell(KB, Percept([Smell, Breeze, Glitter, None, None], 5))

Wumpus — FOL : perceptions et actions

Exemple (suite). Pour déterminer quelle est la meilleure action, l'agent exécute la requête :

$$\text{AskVars}(KB, \exists a \text{ BestAction}(a, 5))$$

(est-ce que KB implique une action particulière en $t = 5$?)

$\text{AskVars}(KB, S)$ retourne certains / tous les σ tels que $KB \models S_\sigma$

Wumpus — FOL : perceptions et actions

Exemple (suite). Pour déterminer quelle est la meilleure action, l'agent exécute la requête :

$$\text{AskVars}(KB, \exists a \text{ BestAction}(a, 5))$$

(est-ce que KB implique une action particulière en $t = 5$?)

Réponse : Oui $\{a/Grab\} \leftarrow$ substitution

Soit une phrase S donnée et une substitution

$$\{\sigma = x_1 / \text{Terme}_1, y_2 / \text{Terme}_2, \dots\}$$

S_σ représente le résultat de la substitution de x_i par Terme_i dans $S \forall i$
 $\text{AskVars}(KB, S)$ retourne certains / tous les σ tels que $KB \models S_\sigma$

Wumpus — FOL : perceptions et actions

Règle de perception : les données perçues impliquent des faits (triviaux)

$$\forall t, s, g, m, c \text{ Percept}([s, \text{Breeze}, g, m, c], t) \Rightarrow \text{Breeze}(t)$$

$$\forall t, s, b, m, c \text{ Percept}([s, b, \text{Glitter}, m, c], t) \Rightarrow \text{Glitter}(t)$$

Remarque : Notez le quantificateur sur le temps t : en logique propositionnelle, nous aurions besoin d'une phrase pour chaque étape de temps.

Wumpus — FOL : perceptions et actions

Un comportement de type “réflexe” peut être exprimé comme suit :

$$\forall t \text{ Glitter}(t) \Rightarrow \text{BestAction}(\text{Grab}, t)$$

Wumpus — FOL : perceptions et actions

Un comportement de type “réflexe” peut être exprimé comme suit :

$$\forall t \text{ Glitter}(t) \Rightarrow \text{BestAction}(\text{Grab}, t)$$

Les **entrées** (perceptions) et **sorties** (actions) d'un agent s'expriment donc facilement grâce à la FOL. Qu'en est-il de l'**environnement** ?

Wumpus — FOL : environnement

Pour exprimer l'**environnement** du monde de Wumpus, quelques **objets** auxquels on pense directement : les cases, les trous et le Wumpus.

On pourrait nommer chaque case (e.g. *Square_{1,2}*) mais alors le fait que les cases *Square_{1,2}* et *Square_{1,3}* sont adjacentes serait un “fait” supplémentaire (et on aurait besoin de tels faits pour toutes les paires de cases adjacentes).

Nous allons plutôt définir un **terme** “liste” dans lequel une case est représentée par deux variables (entières) représentant sa ligne et sa colonne.

Wumpus — FOL : environnement

Les relations d'adjacences peuvent être définies comme suit :

$$\begin{aligned} &\forall x, y, a, b \quad \textit{Adjacent}([x, y], [a, b]) \Leftrightarrow \\ &(x = a \wedge (y = b - 1 \vee y = b + 1)) \vee (y = b \wedge (x = a - 1 \vee x = a + 1)). \end{aligned}$$

Il n'y a pas de raison de distinguer les trous : une propriété suffit

$$\textit{Pit}([x, y]).$$

Pour le Wumpus, unique, une constante est suffisante

$$\textit{Wumpus}.$$

Wumpus — FOL : environnement

La localisation de l'agent change avec le temps, on y fera référence via :

$$At(Agent, [x, y], t).$$

“Le wumpus est en [2,2] et ne bouge pas” :

$$\forall t \quad At(Wumpus, [2, 2], t).$$

“Un objet ne peut pas être à deux endroits en même temps”¹ :

$$\forall x, s_1, s_2, t \quad At(x, s_1, t) \wedge At(x, s_2, t) \Rightarrow s_1 = s_2.$$

1. On abrège la notation $[x, y]$ avec une seule variable pour une case.

Wumpus — FOL : déduire des propriétés cachées

A partir de sa position courante, l'agent peut inférer des propriétés **des cases** à partir des propriétés de ses **perceptions** courantes.

$$\forall s, t \text{ } At(Agent, s, t) \wedge Stench(t) \Rightarrow Smelly(s).$$

$$\forall s, t \text{ } At(Agent, s, t) \wedge Breeze(t) \Rightarrow Breezy(s).$$

Remarque : utile de savoir qu'une case est "*Breezy*" car les trous ne bougent pas (le prédicat *Breezy* n'a donc pas besoin de prendre le temps *t* en argument, mais plutôt une case).

Wumpus — FOL : déduire des propriétés cachées

“Il y a un courant d’air dans une case ssi une case avec un trou est adjacente” :

$$\forall s \text{ Breezy}(s) \Leftrightarrow (\exists r \text{ Pit}(r) \wedge \text{Adjacent}(r, s)).$$

“J’ai une flèche ssi j’en avais une et je ne l’ai pas encore utilisée” :

$$\forall t \text{ HaveArrow}(t+1) \Leftrightarrow (\text{HaveArrow}(t) \wedge \neg \text{Action}(\text{Shoot}, t)).$$

Remarque : ces phrases sont aussi brèves que les phrases équivalentes dans un langage naturel (notre but par rapport au manque d’expressivité de la PL est atteint)

Inférence en Logique du premier ordre

Inférence par énumération en FOL ?

L'implication en logique propositionnelle peut être calculée par **énumération de modèles** (cf. Chap. 6).

On ne pourra plus faire ça ici car l'implication par énumération en FOL est impossible en pratique (il y a trop de modèles), principalement à cause des quantificateurs et des variables !

De la logique du premier ordre (FOL) à la logique propositionnelle (LP)

Nous allons voir comment deux **règles d'inférence** simples

- peuvent être appliquées à des phrases avec quantificateurs pour obtenir des phrases sans quantificateurs ;
- vont permettre de convertir une base de connaissances (KB) en FOL en une KB en LP.

On peut alors (théoriquement) utiliser les méthodes vues au Chapitre 6 pour déduire de nouveaux faits à partir d'une KB en FOL.

Instanciation universelle (Universal Instantiation – UI)

Un exemple très classique de la première règle d'inférence simple (l'instanciation universelle) :

Exemple

Tous les hommes sont mortels. Socrate est un homme. Donc Socrate est mortel.

Instanciation universelle (Universal Instantiation – UI)

Un exemple très classique de la première règle d'inférence simple (l'instanciation universelle) :

Exemple

Tous les hommes sont mortels. Socrate est un homme. Donc Socrate est mortel.

$$\forall x \text{ Human}(x) \Rightarrow \text{Mortal}(x).$$

implique

$$\text{Human}(\text{Socrates}) \Rightarrow \text{Mortal}(\text{Socrates}).$$

Instanciation universelle (Universal Instantiation – UI)

Définition : en FOL, un terme qui ne contient pas de variable est un **terme de base** (ground term).

Règle d'inférence (Instanciation universelle (UI))

Soit $\text{SUBST}(\theta, \alpha)$ la phrase obtenue en appliquant la **substitution** θ à une phrase α (cf. chap. 7). Alors,

$$\frac{\forall v \alpha}{\text{SUBST}(\{v/g\}, \alpha)}$$

pour toute variable v et tout terme de base g .

→ chaque **instanciation** d'une phrase " $\forall v \alpha$ " est impliquée par cette phrase.

Instanciation universelle (Universal Instantiation – UI)

Exemple

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$ implique

Instanciation universelle (Universal Instantiation – UI)

Exemple

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$ implique

$\text{King}(\text{John}) \wedge \text{Greedy}(\text{John}) \Rightarrow \text{Evil}(\text{John})$

$\text{King}(\text{Richard}) \wedge \text{Greedy}(\text{Richard}) \Rightarrow \text{Evil}(\text{Richard})$

$\text{King}(\text{Father}(\text{John})) \wedge \text{Greedy}(\text{Father}(\text{John})) \Rightarrow \text{Evil}(\text{Father}(\text{John}))$

$\vdots$

Instanciation existentielle (Existential instantiation – EI)

Exemple

Quelqu'un a tué la victime. Pour la suite de notre enquête, nous nommerons cette personne le “meurtrier”.

Instanciation existentielle (Existential instantiation – EI)

Exemple

Quelqu'un a tué la victime. Pour la suite de notre enquête, nous nommerons cette personne le “meurtrier”.

$\exists x \text{ Kill}(x, \textit{Victim})$ implique

$\textit{Kill}(\textit{Murderer}, \textit{Victim})$

si la constante *Murderer* n'a pas encore été utilisée.

Cette règle d'inférence simple est l'instanciation existentielle et est un cas particulier de ce qu'on appelle la [skolemization](#).

Instanciation existentielle (Existential instantiation – EI)

Règle d'inférence (Instanciation existentielle (EI))

Pour toute phrase α , toute variable v , et tout symbole constant k qui n'apparaît pas quelque part dans la KB,

$$\frac{\exists v \alpha}{\text{SUBST}(\{v/k\}, \alpha)}$$

→ “ $\exists v \alpha$ ” implique une phrase où la variable v est remplacée par une constante, appelée **constante de Skolem**.

Instanciation existentielle (Existential instantiation – EI)

Exemple

$\exists x \text{ Crown}(x) \wedge \text{OnHead}(x, \text{John})$ implique

$$\text{Crown}(C_1) \wedge \text{OnHead}(C_1, \text{John})$$

à condition que C_1 soit un nouveau symbole.

Exemple

$\exists x \text{ } d(x^y)/dy = x^y$ implique

$$d(e^y)/dy = e^y$$

à condition que e soit un nouveau symbole.

Instanciations universelles et existentielles

- UI peut être appliquée plusieurs fois pour **ajouter** de nouvelles phrases : la nouvelle KB sera équivalente (d'un point de vue logique) à l'ancienne.
- EI peut être appliquée une fois pour **remplacer** la phrase existentielle ($\exists$) : la nouvelle KB **n'est pas** strictement équivalente à l'ancienne, mais est **équivalente au sens de l'inférence**, i.e.
nouvelle KB satisfaisable $\Leftrightarrow$ ancienne KB satisfaisable.

Réduction à la logique propositionnelle

Supposons que KB contienne seulement ceci :

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\text{Greedy}(\text{John})$

$\text{Brother}(\text{Richard}, \text{John})$

Exercice

Appliquez la règle UI à la première phrase de **toutes les manières possibles** ; retirez la phrase universellement quantifiée et donnez la KB obtenue.

Réduction à la logique propositionnelle

Solution : il n'y a que 2 termes de base. On obtient :

King(John) $\wedge$ Greedy(John) $\Rightarrow$ Evil(John)

King(Richard) $\wedge$ Greedy(Richard) $\Rightarrow$ Evil(Richard)

King(John)

Greedy(John)

Brother(Richard, John)

Réduction à la logique propositionnelle

Solution : il n'y a que 2 termes de base. On obtient :

$King(John) \wedge Greedy(John) \Rightarrow Evil(John)$
 $King(Richard) \wedge Greedy(Richard) \Rightarrow Evil(Richard)$
 $King(John)$
 $Greedy(John)$
 $Brother(Richard, John)$

La nouvelle KB est **propositionnalisée** : les phrases atomiques

$King(John)$, $Greedy(John)$, $Evil(John)$, $King(Richard)$, etc.

peuvent être associés à des **symboles de proposition**.

Réduction à la logique propositionnelle

Solution : il n'y a que 2 termes de base. On obtient :

$King(John) \wedge Greedy(John) \Rightarrow Evil(John)$
 $King(Richard) \wedge Greedy(Richard) \Rightarrow Evil(Richard)$
 $King(John)$
 $Greedy(John)$
 $Brother(Richard, John)$

La nouvelle KB est **propositionnalisée** : les phrases atomiques

$King(John)$, $Greedy(John)$, $Evil(John)$, $King(Richard)$, etc.

peuvent être associés à des **symboles de proposition**.

On pourrait donc utiliser les techniques d'inférence du Chap. 6 sur cette KB version LP, et par exemple conclure que $Evil(John)$.

Réduction à la logique propositionnelle

Cette technique de **propositionnalisation** peut être complètement généralisée : toute KB et toute requête q exprimées en FOL peuvent être propositionnalisées de telle sorte que l'**implication soit préservée**.

On a donc une procédure complète pour l'implication en FOL ?

Réduction à la logique propositionnelle

Cette technique de **propositionnalisation** peut être complètement généralisée : toute KB et toute requête q exprimées en FOL peuvent être propositionnalisées de telle sorte que l'**implication soit préservée**.

On a donc une procédure complète pour l'implication en FOL ?

Peut-être pas. . . il y a un **problème** quand KB contient un symbole de fonction (par ex. fonction *Father*) :

Réduction à la logique propositionnelle

Cette technique de **propositionnalisation** peut être complètement généralisée : toute KB et toute requête q exprimées en FOL peuvent être propositionnalisées de telle sorte que l'**implication soit préservée**.

On a donc une procédure complète pour l'implication en FOL ?

Peut-être pas... il y a un **problème** quand KB contient un symbole de fonction (par ex. fonction *Father*) :

L'ensemble des termes de base est infini !

John

Father(John)

Father(Father(John))

Father(Father(Father(John)))

...

Réduction à la logique propositionnelle

Malgré le fait qu'une KB propositionnalisée peut être infinie, le résultat suivant a été prouvé en 1930 !

Théorème (Herbrand)

*Si une phrase α est impliquée par une KB en FOL, alors elle est impliquée par un **sous-ensemble fini** de la KB propositionnalisée.*

→ ce théorème suggère une approche **complète** pour l'inférence en FOL via la propositionnalisation (i.e., toute phrase impliquée peut être trouvée / prouvée)

Réduction à la logique propositionnelle

Approche complète

- tout sous-ensemble fini du théorème précédent a une profondeur maximum pour les termes de base imbriqués
- on peut donc trouver ce sous-ensemble en générant les instances suivantes
 - ▶ d'abord celles avec un symbole constant (*Richard* et *John*)
 - ▶ ensuite celles avec une profondeur 1 (*Father(Richard)* et *Father(John)*)
 - ▶ et ainsi de suite jusqu'à avoir construit une preuve propositionnelle de la phrase α impliquée

Réduction à la logique propositionnelle

Approche complète

- tout sous-ensemble fini du théorème précédent a une profondeur maximum pour les termes de base imbriqués
- on peut donc trouver ce sous-ensemble en générant les instances suivantes
 - ▶ d'abord celles avec un symbole constant (*Richard* et *John*)
 - ▶ ensuite celles avec une profondeur 1 (*Father(Richard)* et *Father(John)*)
 - ▶ et ainsi de suite jusqu'à avoir construit une preuve propositionnelle de la phrase α impliquée

Réduction à la logique propositionnelle

Approche complète

- tout sous-ensemble fini du théorème précédent a une profondeur maximum pour les termes de base imbriqués
- on peut donc trouver ce sous-ensemble en générant les instances suivantes
 - ▶ d'abord celles avec un symbole constant (*Richard* et *John*)
 - ▶ ensuite celles avec une profondeur 1 (*Father(Richard)* et *Father(John)*)
 - ▶ et ainsi de suite jusqu'à avoir construit une preuve propositionnelle de la phrase α impliquée

Réduction à la logique propositionnelle

Approche complète

- tout sous-ensemble fini du théorème précédent a une profondeur maximum pour les termes de base imbriqués
- on peut donc trouver ce sous-ensemble en générant les instances suivantes
 - ▶ d'abord celles avec un symbole constant (*Richard* et *John*)
 - ▶ ensuite celles avec une profondeur 1 (*Father(Richard)* et *Father(John)*)
 - ▶ et ainsi de suite jusqu'à avoir construit une preuve propositionnelle de la phrase α impliquée

Réduction à la logique propositionnelle

Approche complète

- tout sous-ensemble fini du théorème précédent a une profondeur maximum pour les termes de base imbriqués
- on peut donc trouver ce sous-ensemble en générant les instances suivantes
 - ▶ d'abord celles avec un symbole constant (*Richard* et *John*)
 - ▶ ensuite celles avec une profondeur 1 (*Father(Richard)* et *Father(John)*)
 - ▶ et ainsi de suite jusqu'à avoir construit une preuve propositionnelle de la phrase α impliquée

Malheureusement, il reste encore un problème !

Réduction à la logique propositionnelle

Problème : l'approche précédente

- fonctionne si la phrase α est réellement impliquée par la KB,
- ne s'arrête jamais (boucle infinie) si la phrase α n'est pas impliquée par la KB

Réduction à la logique propositionnelle

Problème : l'approche précédente

- fonctionne si la phrase α est réellement impliquée par la KB,
- ne s'arrête jamais (boucle infinie) si la phrase α n'est pas impliquée par la KB

Turing et Church ont chacun prouvé en 1936 (de manière différente) :

Théorème

*L'implication en FOL est **semi décidable**.*

Signification : des algorithmes $\exists$ pour dire “oui” à toute phrase impliquée mais **il n'existe aucun** algorithme capable de dire “non” à toute phrase non impliquée.

Unification et lifting

Phrases non pertinentes

La propositionnalisation génère beaucoup de phrases dont beaucoup semblent non pertinentes.

Exemple

$$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$$

King(John)

$$\forall y \text{ Greedy}(y)$$

Brother(Richard, John)

Il est “évident” que *Evil(John)* mais la propositionnalisation produit beaucoup de faits inutiles pour le prouver (e.g., *Greedy(Richard)*).

Phrases non pertinentes

La propositionnalisation génère beaucoup de phrases dont beaucoup semblent non pertinentes.

Exemple

$$\begin{aligned} &\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x) \\ &\text{King}(\text{John}) \\ &\forall y \text{ Greedy}(y) \\ &\text{Brother}(\text{Richard}, \text{John}) \end{aligned}$$

Il est “évident” que *Evil(John)* mais la propositionnalisation produit beaucoup de faits inutiles pour le prouver (e.g., *Greedy(Richard)*).

Comment rendre cela évident également pour la machine ?

Une règle d'inférence adaptée à FOL

Pour l'exemple précédent, comment inférer directement que *Evil(John)* ?

Il faut trouver une substitution θ qui utilise directement la règle “greedy kings are evil”, i.e., trouver un x tel que *King(x)* et *Greedy(x)*, et en inférer que *Evil(x)*.

Exemple

La substitution $\{x/John\}$ résout la requête *Evil(x)*.

Une règle d'inférence adaptée à FOL

Plus précisément, s'il existe une substitution θ qui rend les prémisses d'une implication **identiques** à des phrases présentes dans la KB, alors on peut inférer la conclusion de l'implication en lui appliquant également la substitution θ .

Exemple

Soit la KB suivante

$$\begin{aligned} &\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x) \\ &\text{King}(\text{John}) \\ &\text{Greedy}(\text{John}) \end{aligned}$$

Alors $\theta = \{x/\text{John}\}$ permet de conclure que $\text{Evil}(\text{John})$ car les prémisses $\text{King}(\text{John})$ et $\text{Greedy}(\text{John})$ sont dans KB.

Une règle d'inférence adaptée à FOL

Question : si au lieu de savoir que *Greedy(John)*, on sait que **tout le monde** est cupide :

$$\forall y \text{ Greedy}(y).$$

Alors, peut-on toujours appliquer cette règle d'inférence pour prouver que *Evil(John)* ?

Une règle d'inférence adaptée à FOL

Question : si au lieu de savoir que *Greedy(John)*, on sait que **tout le monde** est cupide :

$$\forall y \text{ Greedy}(y).$$

Alors, peut-on toujours appliquer cette règle d'inférence pour prouver que *Evil(John)* ?

Réponse : Oui, en appliquant la substitution $\theta = \{x/\text{John}, y/\text{John}\}$ aux prémisses et aux phrases de KB.

A quelle règle d'inférence utilisée au Chap. 6 cette procédure peut faire penser (sauf pour la substitution) ?

Modus Ponens Généralisée (Generalized MP — GMP)

Règle d'inférence (Modus Ponens généralisée)

Soit des phrases atomiques^a p_i , p'_i et q ; et une substitution θ telle que $\text{SUBST}(\theta, p'_i) = \text{SUBST}(\theta, p_i)$ pour tout i . Alors,

$$\frac{p'_1, p'_2, \dots, p'_n, (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{\text{SUBST}(\theta, q)}$$

a. Rappel : phrase atomique = $\text{predicat}(\text{terme}_1, \dots, \text{terme}_n)$ ou $\text{terme}_1 = \text{terme}_2$.

Remarque : on suppose que toutes les variables sont universellement quantifiée ($\forall$).

Modus Ponens Généralisée (Generalized MP — GMP)

Exemple

Soit la KB suivante

$$\begin{aligned} &\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x) \\ &\text{King}(\text{John}) \\ &\forall y \text{ Greedy}(y) \end{aligned}$$

Soit $\theta = \{x/\text{John}, y/\text{John}\}$ et $q = \text{Evil}(x)$. Posons,

p'_i	p_i	$\text{SUBST}(\theta, p'_i) = \text{SUBST}(\theta, p_i)$
$\text{King}(\text{John})$	$\text{King}(x)$	$\text{King}(\text{John})$
$\text{Greedy}(y)$	$\text{Greedy}(x)$	$\text{Greedy}(\text{John})$

Alors GMP implique $\text{Evil}(\text{John})$.

Modus Ponens Généralisée (Generalized MP — GMP)

Rappel :

Règle d'inférence (Modus Ponens)

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

Remarques :

- GMP est une **généralisation** de la règle Modus Ponens (MP, voir Chap. 6) dans le sens où les faits connus et les prémisses d'une implication doivent correspondre "à une substitution près" plutôt que de manière exacte.
- Par contre, MP accepte toute phrase comme prémisse, plutôt que seulement une conjonction de phrases atomiques.
- GMP est une version de MP adaptée ("élevée", **lifted**) à la FOL (plus besoin de propositionnalisation pour appliquer MP). Dans la suite de ce chapitre, on verra qu'on peut faire de même avec le forward / backward chaining par ex.

Modus Ponens Généralisée (Generalized MP — GMP)

Rappel :

Règle d'inférence (Modus Ponens)

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

Remarques :

- GMP est une **généralisation** de la règle Modus Ponens (MP, voir Chap. 6) dans le sens où les faits connus et les prémisses d'une implication doivent correspondre "à une substitution près" plutôt que de manière exacte.
- Par contre, MP accepte toute phrase comme prémisses, plutôt que seulement une conjonction de phrases atomiques.
- GMP est une version de MP adaptée ("élevée", *lifted*) à la FOL (plus besoin de propositionnalisation pour appliquer MP). Dans la suite de ce chapitre, on verra qu'on peut faire de même avec le forward / backward chaining par ex.

Modus Ponens Généralisée (Generalized MP — GMP)

Rappel :

Règle d'inférence (Modus Ponens)

$$\frac{\alpha \Rightarrow \beta, \quad \alpha}{\beta}$$

Remarques :

- GMP est une **généralisation** de la règle Modus Ponens (MP, voir Chap. 6) dans le sens où les faits connus et les prémisses d'une implication doivent correspondre "à une substitution près" plutôt que de manière exacte.
- Par contre, MP accepte toute phrase comme prémisse, plutôt que seulement une conjonction de phrases atomiques.
- GMP est une version de MP adaptée ("élevée", **lifted**) à la FOL (plus besoin de propositionnalisation pour appliquer MP). Dans la suite de ce chapitre, on verra qu'on peut faire de même avec le forward / backward chaining par ex.

Unification

Les règles d'inférences “liftées” nécessitent de **trouver des substitutions** qui font que des expressions logiques différentes apparaissent comme identiques.

Ce processus est appelé l'**unification** (et est primordial pour les algorithmes d'inférence en FOL).

Unification

Les règles d'inférences “liftées” nécessitent de **trouver des substitutions** qui font que des expressions logiques différentes apparaissent comme identiques.

Ce processus est appelé l'**unification** (et est primordial pour les algorithmes d'inférence en FOL).

L'algorithme UNIFY retourne un **unificateur** à partir de deux phrases données.

$\text{UNIFY}(p, q) = \theta$ où $\text{SUBST}(\theta, p) = \text{SUBST}(\theta, q)$.

Unification

Exemple

Soit la requête $AskVars(Knows(John, x))$ ("Qui John connaît-il?").

Réponses à cette question = phrases de KB qui peuvent être unifiées avec $Knows(John, x)$.

Ex. de résultats d'UNIFY avec des phrases qui pourraient être dans KB :

$UNIFY(Knows(John, x), Knows(John, Jane)) =$

$UNIFY(Knows(John, x), Knows(y, Bill)) =$

$UNIFY(Knows(John, x), Knows(y, Mother(y))) =$

$UNIFY(Knows(John, x), Knows(x, Bill)) =$

Unification

Exemple

Soit la requête $AskVars(Knows(John, x))$ ("Qui John connaît-il?").

Réponses à cette question = phrases de KB qui peuvent être unifiées avec $Knows(John, x)$.

Ex. de résultats d'UNIFY avec des phrases qui pourraient être dans KB :

$$UNIFY(Knows(John, x), Knows(John, Jane)) = \{x/Jane\}$$

$$UNIFY(Knows(John, x), Knows(y, Bill)) =$$

$$UNIFY(Knows(John, x), Knows(y, Mother(y))) =$$

$$UNIFY(Knows(John, x), Knows(x, Bill)) =$$

Unification

Exemple

Soit la requête $AskVars(Knows(John, x))$ ("Qui John connaît-il ?").

Réponses à cette question = phrases de KB qui peuvent être unifiées avec $Knows(John, x)$.

Ex. de résultats d'UNIFY avec des phrases qui pourraient être dans KB :

$$\begin{aligned}
 UNIFY(Knows(John, x), Knows(John, Jane)) &= \{x/Jane\} \\
 UNIFY(Knows(John, x), Knows(y, Bill)) &= \{x/Bill, y/John\} \\
 UNIFY(Knows(John, x), Knows(y, Mother(y))) &= \\
 UNIFY(Knows(John, x), Knows(x, Bill)) &=
 \end{aligned}$$

Unification

Exemple

Soit la requête $AskVars(Knows(John, x))$ ("Qui John connaît-il ?").

Réponses à cette question = phrases de KB qui peuvent être unifiées avec $Knows(John, x)$.

Ex. de résultats d'UNIFY avec des phrases qui pourraient être dans KB :

$$\begin{aligned}
 UNIFY(Knows(John, x), Knows(John, Jane)) &= \{x/Jane\} \\
 UNIFY(Knows(John, x), Knows(y, Bill)) &= \{x/Bill, y/John\} \\
 UNIFY(Knows(John, x), Knows(y, Mother(y))) &= \{y/John, x/Mother(John)\} \\
 UNIFY(Knows(John, x), Knows(x, Bill)) &=
 \end{aligned}$$

Unification

Exemple

Soit la requête $AskVars(Knows(John, x))$ ("Qui John connaît-il ?").

Réponses à cette question = phrases de KB qui peuvent être unifiées avec $Knows(John, x)$.

Ex. de résultats d'UNIFY avec des phrases qui pourraient être dans KB :

$UNIFY(Knows(John, x), Knows(John, Jane)) =$	$\{x/Jane\}$
$UNIFY(Knows(John, x), Knows(y, Bill)) =$	$\{x/Bill, y/John\}$
$UNIFY(Knows(John, x), Knows(y, Mother(y))) =$	$\{y/John, x/Mother(John)\}$
$UNIFY(Knows(John, x), Knows(x, Bill)) =$	<i>fail</i>

Unification

Dans l'exemple précédent,

$$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x, \text{Bill})) = \text{fail}$$

car x ne peut prendre à la fois les valeurs John et Bill en même temps.

Pourtant $\text{Knows}(x, \text{Bill})$ signifie “tout le monde connaît Bill”, on devrait donc pouvoir inférer que John connaît Bill.

Unification

Dans l'exemple précédent,

$$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(x, \text{Bill})) = \text{fail}$$

car x ne peut prendre à la fois les valeurs John et Bill en même temps.

Pourtant $\text{Knows}(x, \text{Bill})$ signifie “tout le monde connaît Bill”, on devrait donc pouvoir inférer que John connaît Bill.

Problème : le même nom de variable a été utilisé dans deux phrases différentes.

Solution : **standardiser** les noms des variables.

Unification

Exemple

Soit $p = \text{Knows}(\text{John}, x)$ et $q = \text{Knows}(x, \text{Bill})$.

Après standardisation : $p = \text{Knows}(\text{John}, x_1)$ et $q = \text{Knows}(x_2, \text{Bill})$

$$\text{UNIFY}(p, q) = \{x_1 / \text{Bill}, x_2 / \text{John}\}$$

Unificateurs multiples

Il reste encore un problème ! Que faut-il retourner quand il y a plusieurs unificateurs ?

Unificateurs multiples

Il reste encore un problème ! Que faut-il retourner quand il y a plusieurs unificateurs ?

Exemple

$\text{UNIFY}(\text{Knows}(\text{John}, x), \text{Knows}(y, z))$ peut retourner

$\{y/\text{John}, x/z\}$ ou $\{y/\text{John}, x/\text{John}, z/\text{John}\}$.

Après substitution, le premier unificateur donne $\text{Knows}(\text{John}, z)$ et le second $\text{Knows}(\text{John}, \text{John})$.

Le premier unificateur est **plus général** que le second car il impose moins de restrictions sur les valeurs des variables (le second peut être obtenu à partir du premier via une substitution supplémentaire $\{z/\text{John}\}$).

Unificateur le plus général

Il se fait que pour toute paire d'expressions “unifiables”, il existe un **unificateur le plus général** unique (modulo les noms des variables). On le note **MGU** (most general unifier).

Pour déterminer cet unificateur, un algorithme de complexité **quadratique** en la taille des expressions à unifier existe.

Unificateur le plus général

Idée de l'algorithme UNIFY :

- Explorer récursivement les deux expressions à unifier simultanément, tout en construisant un unificateur.
- L'algorithme échoue (retourne *failure*) si deux entrées du même type (variables, listes, etc.) dans la structure ne correspondent pas.
- Il y a une étape coûteuse (OCCUR-CHECK ?) : quand on veut associer une variable à une phrase, on doit tester si la variable elle-même n'apparaît pas dans la phrase. Autrement l'algorithme pourrait ne pas se terminer (création d'un cycle dans le processus d'unification).

Exemple : x ne peut être unifiée avec $S(x)$.

Unificateur le plus général

Idée de l'algorithme UNIFY :

- Explorer récursivement les deux expressions à unifier simultanément, tout en construisant un unificateur.
- L'algorithme échoue (retourne *failure*) si deux entrées du même type (variables, listes, etc.) dans la structure ne correspondent pas.
- Il y a une étape coûteuse (OCCUR-CHECK ?) : quand on veut associer une variable à une phrase, on doit tester si la variable elle-même n'apparaît pas dans la phrase. Autrement l'algorithme pourrait ne pas se terminer (création d'un cycle dans le processus d'unification).

Exemple : x ne peut être unifiée avec $S(x)$.

→ complexité quadratique (en la taille des expressions) due à ce test.

Unificateur le plus général

```

function UNIFY( $x, y, \theta$ ) returns substitution MGU / failure
  inputs:  $x$ , une variable, constante, liste, ou une expression composée
            $y$ , une variable, constante, liste, ou une expression composée
            $\theta$ , la substitution construite jusqu'à présent (au départ vide)

  if  $\theta$  = failure then return failure
  else if  $x = y$  then return  $\theta$ 
  else if VARIABLE ?( $x$ ) then return UNIFY-VAR( $x, y, \theta$ )
  else if VARIABLE ?( $y$ ) then return UNIFY-VAR( $y, x, \theta$ )
  else if COMPOUND ?( $x$ ) and COMPOUND ?( $y$ ) then
    return UNIFY( $x$ .ARGS,  $y$ .ARGS, UNIFY( $x$ .OP,  $y$ .OP,  $\theta$ ))
  else if LIST ?( $x$ ) and LIST ?( $y$ ) then
    return UNIFY( $x$ .REST,  $y$ .REST, UNIFY( $x$ .FIRST,  $y$ .FIRST,  $\theta$ ))
  else return failure
  
```

Note : pour une expression composée comme $F(A, B)$: le champ $OP \leftarrow F$, et le champ $ARGS \leftarrow$ la liste $[A, B]$

Unificateur le plus général

```

function UNIFY-VAR(var, x,  $\theta$ ) returns a substitution
  inputs: var, une variable
           x, une expression
            $\theta$ , la substitution construite jusqu'à présent

  if {var/val}  $\in \theta$  then return UNIFY(val, x,  $\theta$ )
  else if {x/val}  $\in \theta$  then return UNIFY(var, val,  $\theta$ )
  else if OCCUR-CHECK?(var, x) then return failure
  else add {var/x} to  $\theta$ 
        return  $\theta$ 

```

Unificateur le plus général

Exemple d'application de l'algorithme UNIFY

UNIFY(*Knows*(*John*, *x*), *Knows*(*y*, *z*), $\{\}$) : 2 expressions composées

- UNIFY([*John*, *x*], [*y*, *z*], UNIFY(*Knows*, *Knows*, $\{\}$))
 - ▶ UNIFY(*Knows*, *Knows*, $\{\}$) : 2 expressions identiques
 - ▶ retourne $\theta = \{\}$
- UNIFY([*John*, *x*], [*y*, *z*], $\{\}$) : 2 listes
 - ▶ UNIFY([*x*], [*z*], UNIFY(*John*, *y*, $\{\}$))
 - ▶ UNIFY(*John*, *y*, $\{\}$) : UNIFY-VAR(*John*, *y*, $\{\}$)
 - ▶ retourne $\theta = \{y/John\}$
 - ▶ UNIFY([*x*], [*z*], $\{y/John\}$) : UNIFY-VAR(*x*, *z*, $\{y/John\}$)
 - ▶ retourne $\theta = \{y/John, x/z\}$
- retourne $\theta = \{y/John, x/z\}$

Unificateur le plus général

Exemple d'application de l'algorithme UNIFY

UNIFY(*Knows*(*John*, *x*), *Knows*(*y*, *z*), $\{\}$) : 2 expressions composées

- UNIFY([*John*, *x*], [*y*, *z*], UNIFY(*Knows*, *Knows*, $\{\}$))
 - ▶ UNIFY(*Knows*, *Knows*, $\{\}$) : 2 expressions identiques
 - ▶ retourne $\theta = \{\}$
- UNIFY([*John*, *x*], [*y*, *z*], $\{\}$) : 2 listes
 - ▶ UNIFY([*x*], [*z*], UNIFY(*John*, *y*, $\{\}$))
 - ▶ UNIFY(*John*, *y*, $\{\}$) : UNIFY-VAR(*y*, *John*, $\{\}$)
 - ▶ retourne $\theta = \{y/John\}$
 - ▶ UNIFY([*x*], [*z*], $\{y/John\}$) : UNIFY-VAR(*x*, *z*, $\{y/John\}$)
 - ▶ retourne $\theta = \{y/John, x/z\}$
- retourne $\theta = \{y/John, x/z\}$

Unificateur le plus général

Exemple d'application de l'algorithme UNIFY

UNIFY(*Knows*(*John*, *x*), *Knows*(*y*, *z*), $\{\}$) : 2 expressions composées

- UNIFY([*John*, *x*], [*y*, *z*], UNIFY(*Knows*, *Knows*, $\{\}$))
 - ▶ UNIFY(*Knows*, *Knows*, $\{\}$) : 2 expressions identiques
 - ▶ retourne $\theta = \{\}$
- UNIFY([*John*, *x*], [*y*, *z*], $\{\}$) : 2 listes
 - ▶ UNIFY([*x*], [*z*], UNIFY(*John*, *y*, $\{\}$))
 - ▶ UNIFY(*John*, *y*, $\{\}$) : UNIFY-CONST(*John*, *y*, $\{\}$)
 - ▶ retourne $\theta = \{y/John\}$
 - ▶ UNIFY([*x*], [*z*], $\{y/John\}$) : UNIFY-VAR(*x*, *z*, $\{y/John\}$)
 - ▶ retourne $\theta = \{y/John, x/z\}$
- retourne $\theta = \{y/John, x/z\}$

Unificateur le plus général

Exemple d'application de l'algorithme UNIFY

UNIFY(*Knows*(John, x), *Knows*(y, z), {}) : 2 expressions composées

- UNIFY([John, x], [y, z], UNIFY(*Knows*, *Knows*, {}))
 - ▶ UNIFY(*Knows*, *Knows*, {}) : 2 expressions identiques
 - ▶ retourne $\theta = \{\}$
- UNIFY([John, x], [y, z], {}) : 2 listes
 - ▶ UNIFY([x], [z], UNIFY(*John*, y, {}))
 - ★ UNIFY(*John*, y, {}) : UNIFY-VAR(y, *John*, {})
 - ★ retourne $\theta = \{y/\text{John}\}$
 - ▶ UNIFY([x], [z], {y/*John*}) : UNIFY-VAR(x, z, {y/*John*})
 - ▶ retourne $\theta = \{y/\text{John}, x/z\}$
- retourne $\theta = \{y/\text{John}, x/z\}$

Unificateur le plus général

Exemple d'application de l'algorithme UNIFY

UNIFY(*Knows*(*John*, *x*), *Knows*(*y*, *z*), $\{\}$) : 2 expressions composées

- UNIFY([*John*, *x*], [*y*, *z*], UNIFY(*Knows*, *Knows*, $\{\}$))
 - ▶ UNIFY(*Knows*, *Knows*, $\{\}$) : 2 expressions identiques
 - ▶ retourne $\theta = \{\}$
- UNIFY([*John*, *x*], [*y*, *z*], $\{\}$) : 2 listes
 - ▶ UNIFY([*x*], [*z*], UNIFY(*John*, *y*, $\{\}$))
 - ★ UNIFY(*John*, *y*, $\{\}$) : UNIFY-VAR(*y*, *John*, $\{\}$)
 - ★ retourne $\theta = \{y/John\}$
 - ▶ UNIFY([*x*], [*z*], $\{y/John\}$) : UNIFY-VAR(*x*, *z*, $\{y/John\}$)
 - ▶ retourne $\theta = \{y/John, x/z\}$
- retourne $\theta = \{y/John, x/z\}$

Unificateur le plus général

Exemple d'application de l'algorithme UNIFY

UNIFY(*Knows*(*John*, *x*), *Knows*(*y*, *z*), $\{\}$) : 2 expressions composées

- UNIFY([*John*, *x*], [*y*, *z*], UNIFY(*Knows*, *Knows*, $\{\}$))
 - ▶ UNIFY(*Knows*, *Knows*, $\{\}$) : 2 expressions identiques
 - ▶ retourne $\theta = \{\}$
- UNIFY([*John*, *x*], [*y*, *z*], $\{\}$) : 2 listes
 - ▶ UNIFY([*x*], [*z*], UNIFY(*John*, *y*, $\{\}$))
 - ★ UNIFY(*John*, *y*, $\{\}$) : UNIFY-VAR(*y*, *John*, $\{\}$)
 - ★ retourne $\theta = \{y/John\}$
 - ▶ UNIFY([*x*], [*z*], $\{y/John\}$) : UNIFY-VAR(*x*, *z*, $\{y/John\}$)
 - ▶ retourne $\theta = \{y/John, x/z\}$
- retourne $\theta = \{y/John, x/z\}$

Unificateur le plus général

Exemple d'application de l'algorithme UNIFY

UNIFY(*Knows*(*John*, *x*), *Knows*(*y*, *z*), $\{\}$) : 2 expressions composées

- UNIFY([*John*, *x*], [*y*, *z*], UNIFY(*Knows*, *Knows*, $\{\}$))
 - ▶ UNIFY(*Knows*, *Knows*, $\{\}$) : 2 expressions identiques
 - ▶ retourne $\theta = \{\}$
- UNIFY([*John*, *x*], [*y*, *z*], $\{\}$) : 2 listes
 - ▶ UNIFY([*x*], [*z*], UNIFY(*John*, *y*, $\{\}$))
 - ★ UNIFY(*John*, *y*, $\{\}$) : UNIFY-VAR(*y*, *John*, $\{\}$)
 - ★ retourne $\theta = \{y/John\}$
 - ▶ UNIFY([*x*], [*z*], $\{y/John\}$) : UNIFY-VAR(*x*, *z*, $\{y/John\}$)
 - ▶ retourne $\theta = \{y/John, x/z\}$
- retourne $\theta = \{y/John, x/z\}$

Unificateur le plus général

Exemple d'application de l'algorithme UNIFY

UNIFY(*Knows*(*John*, *x*), *Knows*(*y*, *z*), $\{\}$) : 2 expressions composées

- UNIFY([*John*, *x*], [*y*, *z*], UNIFY(*Knows*, *Knows*, $\{\}$))
 - ▶ UNIFY(*Knows*, *Knows*, $\{\}$) : 2 expressions identiques
 - ▶ retourne $\theta = \{\}$
- UNIFY([*John*, *x*], [*y*, *z*], $\{\}$) : 2 listes
 - ▶ UNIFY([*x*], [*z*], UNIFY(*John*, *y*, $\{\}$))
 - ★ UNIFY(*John*, *y*, $\{\}$) : UNIFY-VAR(*y*, *John*, $\{\}$)
 - ★ retourne $\theta = \{y/John\}$
 - ▶ UNIFY([*x*], [*z*], $\{y/John\}$) : UNIFY-VAR(*x*, *z*, $\{y/John\}$)
 - ▶ retourne $\theta = \{y/John, x/z\}$
- retourne $\theta = \{y/John, x/z\}$

Forward chaining

Algorithme de Forward chaining pour FOL

Nous allons “élever” le FC à FOL en définissant des **clauses définies du premier ordre**.

Une **clause définie du premier ordre** est semblable à une clause définie propositionnelle = **disjonction de littéraux dont un seul est positif**.

En FOL, une clause définie peut contenir des variables (univers. quantif.). Comme en LP, on observe qu'on peut les écrire de 2 façons :

- soit une phrase **atomique** ;
- soit une implication dont l'antécédent est une **conjonction de littéraux positifs** et dont la conséquence est **un seul littéral positif**.

Algorithme de Forward chaining pour FOL

Nous allons “élever” le FC à FOL en définissant des **clauses définies** du premier ordre.

Une **clause définie du premier ordre** est semblable à une clause définie propositionnelle = **disjonction de littéraux dont un seul est positif**.

En FOL, une clause définie peut contenir des variables (univers. quantif.). Comme en LP, on observe qu'on peut les écrire de 2 façons :

- soit une phrase **atomique** ;
- soit une implication dont l'antécédent est une **conjonction de littéraux positifs** et dont la conséquence est **un seul littéral positif**.

Exemple

$King(x) \wedge Greedy(x) \Rightarrow Evil(x).$
 $King(John).$
 $Greedy(y).$

Exemple de KB

Toutes les KB ne peuvent pas être converties en ensemble de clauses définies, mais beaucoup le peuvent.

Exercice

Soit la connaissance suivante. Est-il possible de la décrire sous la forme de clauses définies du premier ordre ?

La loi dit que vendre des armes à des nations hostiles est un crime pour un belge. Le pays Nono, un ennemi de la Belgique, possède des missiles et tous ses missiles ont été vendus par le Colonel Flupke, qui est Belge.

Exemple de KB

- “vendre des armes à des nations hostiles est un crime pour un belge.”

$$Belgian(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x). \quad (1)$$

- “Nono (...) possède des missiles” (i.e., $\exists x Owns(Nono, x) \wedge Missile(x)$)

$$Owns(Nono, M_1). \quad (2)$$

$$Missile(M_1). \quad (3)$$

Exemple de KB

- “vendre des armes à des nations hostiles est un crime pour un belge.”

$$Belgian(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x). \quad (1)$$

- “Nono (...) possède des missiles” (i.e., $\exists x Owns(Nono, x) \wedge Missile(x)$)

$$Owns(Nono, M_1). \quad (2)$$

$$Missile(M_1). \quad (3)$$

Exemple de KB

- “tous ses missiles ont été vendus par le Colonel Flupke”

$$Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(Flupke, x, Nono). \quad (4)$$

- Nous avons aussi besoin de savoir qu’une missile est une arme (pas dit explicitement) :

$$Missile(x) \Rightarrow Weapon(x). \quad (5)$$

- Et nous devons savoir qu’un ennemi de la Belgique est “hostile” :

$$Enemy(x, Belgium) \Rightarrow Hostile(x). \quad (6)$$

Exemple de KB

- “tous ses missiles ont été vendus par le Colonel Flupke”

$$Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(Flupke, x, Nono). \quad (4)$$

- Nous avons aussi besoin de savoir qu’une missile est une arme (pas dit explicitement) :

$$Missile(x) \Rightarrow Weapon(x). \quad (5)$$

- Et nous devons savoir qu’un ennemi de la Belgique est “hostile” :

$$Enemy(x, Belgium) \Rightarrow Hostile(x). \quad (6)$$

Exemple de KB

- “tous ses missiles ont été vendus par le Colonel Flupke”

$$Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(Flupke, x, Nono). \quad (4)$$

- Nous avons aussi besoin de savoir qu’une missile est une arme (pas dit explicitement) :

$$Missile(x) \Rightarrow Weapon(x). \quad (5)$$

- Et nous devons savoir qu’un ennemi de la Belgique est “hostile” :

$$Enemy(x, Belgium) \Rightarrow Hostile(x). \quad (6)$$

Exemple de KB

- “Flupke, qui est Belge”

Belgian(Flupke). (7)

- “Le pays Nono, un ennemi de la Belgique”

Enemy(Nono, Belgium) (8)

2. Datalog = uniquement clauses définies en FOL et pas de fonctions. Ce langage tire son nom du fait qu'il peut représenter le type d'assertion typiquement utilisée dans des bases de données relationnelles.

Exemple de KB

- “Flupke, qui est Belge”

Belgian(Flupke). (7)

- “Le pays Nono, un ennemi de la Belgique”

Enemy(Nono, Belgium) (8)

2. Datalog = uniquement clauses définies en FOL et pas de fonctions. Ce langage tire son nom du fait qu'il peut représenter le type d'assertion typiquement utilisée dans des bases de données relationnelles.

Exemple de KB

- “Flupke, qui est Belge”

Belgian(Flupke). (7)

- “Le pays Nono, un ennemi de la Belgique”

Enemy(Nono, Belgium) (8)

→ la KB contient 8 clauses définies (numérotées de (1) à (8)) et ne contient pas de symboles de fonctions : c’est ce qu’on appelle une KB **Datalog**².

2. Datalog = uniquement clauses définies en FOL et pas de fonctions. Ce langage tire son nom du fait qu’il peut représenter le type d’assertion typiquement utilisée dans des bases de données relationnelles.

Algorithme de Forward chaining pour FOL

L'algorithme FOL-FC-Ask (slide suivant)

- prend en entrée
 - ▶ une KB ne contenant que des clauses définies du premier ordre
 - ▶ une requête α qui est une phrase atomique en FOL
- utilise une variable locale *new* qui est un ensemble (initialement vide) de nouvelles phrases inférées par le FC

Algorithme de Forward chaining pour FOL

```

function FOL-FC-Ask( $KB, \alpha$ ) returns a substitution or false

  repeat until new is empty
     $new \leftarrow \{\}$ 
    for each rule in  $KB$  do
       $(p_1 \wedge \dots \wedge p_n \Rightarrow q) \leftarrow \text{STANDARDIZE-VARIABLES}(\text{rule})$ 
      for each  $\theta$  such that  $\text{SUBST}(\theta, p_1 \wedge \dots \wedge p_n) = \dots$ 
         $\text{SUBST}(\theta, p'_1 \wedge \dots \wedge p'_n)$  for some  $p'_1, \dots, p'_n$  in  $KB$ 
       $q' \leftarrow \text{SUBST}(\theta, q)$ 
      if  $q'$  does not unify with some sentence already in  $KB \dots$ 
        or new then
          add  $q'$  to new
           $\phi \leftarrow \text{UNIFY}(q', \alpha)$ 
          if  $\phi$  is not fail then return  $\phi$ 
    add new to  $KB$ 
  return false

```

Preuve par Forward Chaining

Exercice

Prouver que le Colonel Flupke est un criminel (par Forward Chaining).

KB = les clauses définies (1) à (8)

$\alpha = \textit{Criminal}(x)$

Preuve par Forward Chaining

Faits :

(F1) *Owns(Nono, M₁)*

(F2) *Missile(M₁)*

(F3) *Belgian(Flupke)*

(F4) *Enemy(Nono, Belgium)*

(F5) *Sells(Flupke, M₁, Nono)*

(F6) *Weapon(M₁)*

(F7) *Hostile(Nono)*

(F8) *Criminal(Flupke)*

- La KB est constituée de 4 faits (F1 à F4) et 4 implications (cf. (1), (4), (5) et (6))
- Première itération de FOL-FC-Ask :
 - ▶ Prémisses de (1) non satisfaits : *Belgian(x) ∧ Weapon(y) ∧ ...*
 - ▶ $\{x/M_1\}$ satisfait (4) : F5 ajouté
 - ▶ $\{x/M_1\}$ satisfait (5) : F6 ajouté
 - ▶ $\{x/Nono\}$ satisfait (6) : F7 ajouté
- Deuxième itération de FOL-FC-Ask :
 - ▶ $\{x/Flupke, y/M_1, z/Nono\}$ satisfait (1) : F8 ajouté (et preuve terminée)

Preuve par Forward Chaining

Faits :

(F1) *Owns(Nono, M₁)*

(F2) *Missile(M₁)*

(F3) *Belgian(Flupke)*

(F4) *Enemy(Nono, Belgium)*

(F5) *Sells(Flupke, M₁, Nono)*

(F6) *Weapon(M₁)*

(F7) *Hostile(Nono)*

(F8) *Criminal(Flupke)*

- La KB est constituée de 4 faits (F1 à F4) et 4 implications (cf. (1), (4), (5) et (6))

- Première itération de FOL-FC-Ask :

- ▶ Prémisses de (1) non satisfaits :

Belgian(x) ∧ Weapon(y) ∧ ...

- ▶ $\{x/M_1\}$ satisfait (4) : F5 ajouté

- ▶ $\{x/M_1\}$ satisfait (5) : F6 ajouté

- ▶ $\{x/Nono\}$ satisfait (6) : F7 ajouté

- Deuxième itération de FOL-FC-Ask :

- ▶ $\{x/Flupke, y/M_1, z/Nono\}$ satisfait (1) : F8 ajouté (et preuve terminée)

$$Belgian(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x) \quad (1)$$

Preuve par Forward Chaining

Faits :

(F1) *Owns(Nono, M₁)*

(F2) *Missile(M₁)*

(F3) *Belgian(Flupke)*

(F4) *Enemy(Nono, Belgium)*

(F5) *Sells(Flupke, M₁, Nono)*

(F6) *Weapon(M₁)*

(F7) *Hostile(Nono)*

(F8) *Criminal(Flupke)*

- La KB est constituée de 4 faits (F1 à F4) et 4 implications (cf. (1), (4), (5) et (6))

- Première itération de FOL-FC-Ask :

- ▶ Prémisses de (1) non satisfaits :

Belgian(x) ∧ Weapon(y) ∧ ...

- ▶ $\{x/M_1\}$ satisfait (4) : F5 ajouté

- ▶ $\{x/M_1\}$ satisfait (5) : F6 ajouté

- ▶ $\{x/Nono\}$ satisfait (6) : F7 ajouté

- Deuxième itération de FOL-FC-Ask :

- ▶ $\{x/Flupke, y/M_1, z/Nono\}$ satisfait (1) : F8 ajouté (et preuve terminée)

$$Missile(x) \wedge Owns(Nono, x) \Rightarrow Sells(Flupke, x, Nono) \quad (4)$$

Preuve par Forward Chaining

Faits :

(F1) *Owns(Nono, M₁)*

(F2) *Missile(M₁)*

(F3) *Belgian(Flupke)*

(F4) *Enemy(Nono, Belgium)*

(F5) *Sells(Flupke, M₁, Nono)*

(F6) *Weapon(M₁)*

(F7) *Hostile(Nono)*

(F8) *Criminal(Flupke)*

- La KB est constituée de 4 faits (F1 à F4) et 4 implications (cf. (1), (4), (5) et (6))

- Première itération de FOL-FC-Ask :

- ▶ Prémisses de (1) non satisfaits :

Belgian(x) ∧ Weapon(y) ∧ ...

- ▶ $\{x/M_1\}$ satisfait (4) : F5 ajouté

- ▶ $\{x/M_1\}$ satisfait (5) : F6 ajouté

- ▶ $\{x/Nono\}$ satisfait (6) : F7 ajouté

- Deuxième itération de FOL-FC-Ask :

- ▶ $\{x/Flupke, y/M_1, z/Nono\}$ satisfait (1) : F8 ajouté (et preuve terminée)

$$Missile(x) \Rightarrow Weapon(x) \quad (5)$$

Preuve par Forward Chaining

Faits :

(F1) *Owns(Nono, M₁)*

(F2) *Missile(M₁)*

(F3) *Belgian(Flupke)*

(F4) *Enemy(Nono, Belgium)*

(F5) *Sells(Flupke, M₁, Nono)*

(F6) *Weapon(M₁)*

(F7) *Hostile(Nono)*

(F8) *Criminal(Flupke)*

- La KB est constituée de 4 faits (F1 à F4) et 4 implications (cf. (1), (4), (5) et (6))
- Première itération de FOL-FC-Ask :
 - ▶ Prémisses de (1) non satisfaits : *Belgian(x) ∧ Weapon(y) ∧ ...*
 - ▶ $\{x/M_1\}$ satisfait (4) : F5 ajouté
 - ▶ $\{x/M_1\}$ satisfait (5) : F6 ajouté
 - ▶ $\{x/Nono\}$ satisfait (6) : F7 ajouté
- Deuxième itération de FOL-FC-Ask :
 - ▶ $\{x/Flupke, y/M_1, z/Nono\}$ satisfait (1) : F8 ajouté (et preuve terminée)

$$Enemy(x, Belgium) \Rightarrow Hostile(x) \quad (6)$$

Preuve par Forward Chaining

Faits :

- (F1) *Owns(Nono, M₁)*
- (F2) *Missile(M₁)*
- (F3) *Belgian(Flupke)*
- (F4) *Enemy(Nono, Belgium)*
- (F5) *Sells(Flupke, M₁, Nono)*
- (F6) *Weapon(M₁)*
- (F7) *Hostile(Nono)*
- (F8) *Criminal(Flupke)*

- La KB est constituée de 4 faits (F1 à F4) et 4 implications (cf. (1), (4), (5) et (6))
- Première itération de FOL-FC-Ask :
 - ▶ Prémisses de (1) non satisfaits : *Belgian(x) ∧ Weapon(y) ∧ ...*
 - ▶ $\{x/M_1\}$ satisfait (4) : F5 ajouté
 - ▶ $\{x/M_1\}$ satisfait (5) : F6 ajouté
 - ▶ $\{x/Nono\}$ satisfait (6) : F7 ajouté
- Deuxième itération de FOL-FC-Ask :
 - ▶ $\{x/Flupke, y/M_1, z/Nono\}$ satisfait (1) : F8 ajouté (et preuve terminée)

$$Belgian(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x) \quad (1)$$

Preuve par Forward Chaining

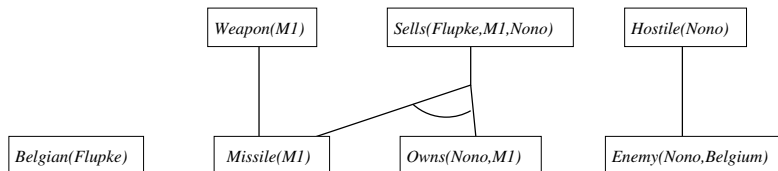
Belgian(Flupke)

Missile(M1)

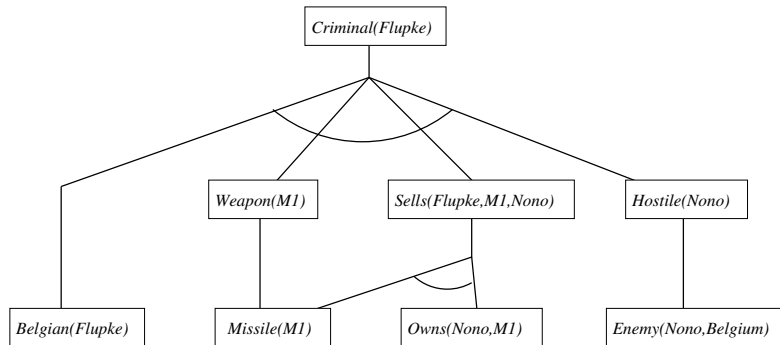
Owns(Nono,M1)

Enemy(Nono,Belgium)

Preuve par Forward Chaining



Preuve par Forward Chaining



Propriétés of forward chaining

Sensé et complet pour les clauses définies du premier ordre
(preuve similaire à la preuve propositionnelle)

Datalog = clauses définies du premier ordre + **pas de fonctions** (e.g., KB “crime”) :

Temps polynomial pour FC avec Datalog : en effet, il y a au plus $p \cdot n^k$ faits ajoutés³.

En général (avec fonctions) : peut ne pas se terminer si α n'est pas impliquée (c'est inévitable cf. théorème vu ci-avant).

3. Où k = nombre maximum d'arguments pour un prédicat, p = nombre de prédicats et n = nombre de symboles constants. Il y a au max. n^k substitutions possibles pour chaque prédicat.

Backward chaining et Programmation logique

Backward chaining

Le BC peut également être élevé à FOL (non couvert dans ce cours, voir livre de référence).

Le BC est à la base de la programmation logique.

Programmation logique

Programmation logique = inférence sur des KBs logiques

Programmation logique	Programmation classique
1. Identifier le problème	Identifier le problème
2. Assembler l'information	Assembler l'information
3. Pause café	Trouver une solution
4. Coder l'information dans KB	Programmer la solution
5. Encoder une instance du problème avec des faits	Encoder une instance du problème avec des données
6. Soumettre des requêtes	Appliquer le programme aux données
7. Trouver des faits qui sont faux	Deboguer les fonctions / méthodes

Systèmes Prolog

Prolog :

- langage de programmation logique le plus répandu
- très populaire en IA
- **programme** prolog = ensemble de clauses définies
- **notation** : à l'opposé des conventions utilisés en logique (!)

Systèmes Prolog

Syntaxe Prolog :

- majuscule pour une **variable** et minuscule (ou chiffre) pour un symbole constant ou un prédicat
- une clause se termine par un point
- **virgule** à la place de $\wedge$
- clause définie écrite “à l’envers” : $A \wedge B \Rightarrow C$ devient :

$C \text{ :- } A, B.$

Exemple

```
criminal(X) :- belgian(X), weapon(Y), sells(X,Y,Z), hostile(Z).  
belgian(flupke).
```

Systèmes Prolog

Sémantique Prolog $\neq$ FOL car impose 3 hypothèses :

- **Noms uniques** : tout symbole constant réfère à un objet distinct (e.g. *John* $\neq$ *Richard*).
- **monde clos** : une phrase atomique dont on ne sait pas si elle est vraie est considérée comme un fait **faux**.
- **fermeture du domaine** : chaque modèle ne contient pas plus d'éléments du domaine que ceux qui sont nommés par les symboles constants.

→ appelé la sémantique “base de données”

⇒ Prolog est moins expressif que FOL mais rend l'inférence plus simple.

Systèmes Prolog

Nombreuses implémentations de prolog.

Exemple : <http://www.swi-prolog.org/> (Open source, Université d'Amsterdam)

→ Voir démonstrations ci-après.

Exemples de programmes Prolog

Exemple

Soit le fichier `crime.pl` contenant la KB suivante :

```
criminal(X) :- belgian(X), weapon(Y), sells(X, Y, Z), hostile(Z).  
owns(nono, m1).  
missile(m1).  
sells(flupke, X, nono) :- missile(X), owns(nono, X).  
belgian(flupke).  
belgian(joe).  
hostile(X) :- ennemy(X, belgium).  
ennemy(nono, belgium).  
weapon(X) :- missile(X).  
sunny.
```

Exemples de programmes Prolog

Exemple (crime.pl – session swipl 1/2)

```
machine$ swipl
Welcome to SWI-Prolog (Multi-threaded, 32 bits, Version 5.10.2)
...
?- [crime].
% crime compiled 0.00 sec, 2,396 bytes
true.

?- sunny.
true.

?- belgian(joe).
true.
```

Exemples de programmes Prolog

Exemple (crime.pl – session swipl 2/2)

```
?- belgian(X).
```

```
X = flupke ;
```

```
X = joe.
```

```
?- criminal(X).
```

```
X = flupke ;
```

```
false.
```

```
?- criminal(joe).
```

```
false.
```

```
?- halt.
```

```
machine$
```

Exemples de programmes Prolog

Exemple

Soit le fichier `likes.pl` contenant la KB suivante :

```
likes(sam, Food) :- indian(Food), mild(Food).
```

```
likes(sam, Food) :- chinese(Food).
```

```
likes(sam, Food) :- italian(Food).
```

```
likes(sam, chips).
```

```
indian(curry).
```

```
indian(tandoori).
```

```
indian(kurma).
```

```
mild(kurma).
```

```
chinese(chop_suey).
```

```
chinese(sweet_and_sour).
```

```
italian(pizza).
```

```
italian(spaghetti).
```

Exemples de programmes Prolog

Exemple (crime.pl – session swipl)

```
?- [likes].  
% likes compiled 0.00 sec, 2,028 bytes  
true.
```

```
?- likes(X,Y).  
X = sam,  
Y = kurma ;  
X = sam,  
Y = chop_suey ;  
X = sam,  
Y = sweet_and_sour ;  
X = sam,  
Y = pizza ;  
X = sam,  
Y = spaghetti ;  
X = sam,  
Y = chips.
```

Preuves par Résolution

Preuves par Résolution

Au Chap. 6 : méthode de preuve par résolution (qui utilise l'idée d'une **preuve par l'absurde**) → procédure d'inférence complète en logique propositionnelle

Il est possible d'étendre la règle de résolution à FOL en transformant chaque phrase en **forme normale conjonctive** (cf. AIMA, non vu dans ce cours).

La règle d'inférence par résolution généralisée est **complète** en FOL.

Résumé du chapitre

Résumé du chapitre

Logique du premier ordre (FOL) :

- les objets et relations sont les éléments primitifs de la sémantique
- syntaxe : constantes, fonctions, prédicats, égalité, quantificateurs

Puissance d'expression augmentée : suffisante pour décrire les relations familiales, la théorie des ensembles, les listes, le monde de Wumpus, etc.

Résumé du chapitre

Méthodes d'inférence en FOL.

- Une première approche consiste à utiliser deux règles d'inférence (instanciations universelle et existentielle) pour **propositionnaliser** le problème d'inférence
- L'utilisation de l'**unification** pour identifier des substitutions appropriées permet d'élever des règles en FOL sans devoir propositionnaliser (gain d'efficacité)
- Ainsi, la règle **Modus Ponens généralisée** (GMP) utilise l'unification pour élever la règle Modus Ponens à FOL.
- Les algorithmes FC et BC, ainsi que la règle de résolution peuvent également être élevés à FOL.

Résumé du chapitre

- GMP est complet pour les clauses définies mais l'implication est **semidécidable**. Cependant, pour des KB sans fonctions (Datalog), l'implication est décidable.
- FC est utilisé dans les **bases de données déductives**.
- BC est utilisé en **programmation logique**.
- **Prolog**, contrairement à la logique du premier ordre, utilise un monde clos avec un domaine fermé et des noms uniques : cela le rend plus pratique comme langage de programmation mais diminue l'expressivité.